

UpDesk Fork Modifications

Questo documento descrive le modifiche presenti nel fork UpDesk costruito a partire da RustDesk 1.4.6.

Scopo del documento:

- spiegare cosa è stato cambiato rispetto alla base RustDesk
- indicare dove si trovano le modifiche nel repository
- separare chiaramente le personalizzazioni di prodotto dalle integrazioni server/deploy
- lasciare una base tecnica leggibile per manutenzione, audit e passaggio di consegne

Nota importante:

- questo documento è ricostruito dallo stato attuale del repository
- non è una cronologia commit-per-commit
- dove la storia esatta non è ricavabile senza git history, il documento descrive il risultato tecnico consolidato

1. Baseline del fork

Base di partenza:

- client RustDesk 1.4.6
- server relay/rendezvous di riferimento incluso localmente:
- third_party/updesk-server-1.1.15

Obiettivi principali del fork:

- rebrand completo come UpDesk / UptimeDesk
- controllo del relay self-hosted su updesk.uptimeservice.it
- compatibilità con reti restrittive tramite fallback websocket su 443
- packaging e deploy proprietari
- primo sistema di auto-update professionale
- UI e policy più vicine a un prodotto commerciale

2. Rebranding del prodotto

Il fork non è solo un cambio nome superficiale: l'applicazione, i binari, il packaging e parte del testo UI sono stati riallineati a UpDesk / UptimeDesk.

2.1 Nomi binari e crate

File principali:

- Cargo.toml
- build.py
- src/version.rs

Modifiche:

- crate principale rinominato in uptimedesk

- libreria desktop rinominata in libuptimedesk
- helper updater separato aggiunto come binario:
- updesk_updater
- metadati Windows/macOS/Linux aggiornati:
- ProductName = "UpDesk"
- FileDescription = "UpDesk Remote Desktop"
- identificatori app e nomi bundle personalizzati

2.2 Branding UI e testi

Evidenze:

- riferimenti UpDesk / UptimeDesk diffusi in:
- flutter/lib/...
- src/lang/.rs
- src/platform/...
- build.py

Modifiche:

- nome prodotto mostrato nelle viste desktop
- testi di installazione, update, sicurezza e sessione riallineati al brand
- diverse traduzioni aggiornate per sostituire RustDesk con UptimeDesk

2.3 Asset e packaging

File rilevanti:

- uptimedesk_setup.iss
- uptimedesk_full_setup.iss
- build.py
- appimage/AppImageBuilder-x86_64.yml
- appimage/AppImageBuilder-aarch64.yml

Modifiche:

- nomi installer Windows personalizzati
- naming dei pacchetti Linux e macOS personalizzato
- packaging AppImage e bundle desktop rinominati

3. Architettura relay e fallback 443

Questa è la modifica strutturale più importante del fork.

Obiettivo:

- far funzionare UpDesk anche quando le porte standard RustDesk sono bloccate in uscita
- mantenere hbbs/hbbr 1.1.15
- usare nginx e websocket 443 senza riscrivere l'intera architettura server

Documentazione tecnica dedicata:

- docs/updesk-relay-compatible.md

3.1 Topologia del relay custom

Host:

- updesk.uptimeservice.it

Path pubblici:

- https://updesk.uptimeservice.it/ws/id
- https://updesk.uptimeservice.it/ws/relay

Instradamento:

- /ws/id -> bridge compat websocket rendezvous
- /ws/relay -> relay pair proxy websocket
- 21116/tcp -> proxy compat hbbs
- 21117/tcp -> proxy compat relay TCP

3.2 File server-side aggiunti

File chiave:

- server/ws_hbbs_bridge.py
- server/hbbs_tcp_proxy.py
- server/relay_pair_proxy.py
- server/updesk-bridge.service
- server/updesk-relay-pair.service
- server/updesk_nginx_patch.py

3.3 Problemi risolti

a. Mismatch protobuf client/server

Caso:

- client 1.4.6:
- RequestRelay = 18
- RelayResponse = 19
- hbbs 1.1.15:
- RequestRelay = 9
- RelayResponse = 10

Soluzione:

- bridge/proxy che traducono i field protobuf in modo trasparente

b. Compatibilita websocket rendezvous

Il bridge ws_hbbs_bridge.py gestisce:

- RegisterPk
- RegisterPeer
- OnlineRequest
- OnlineResponse
- PunchHoleRequest

- RelayResponse
- RequestRelay

in modo compatibile con client nuovo e server vecchio

c. Relay pairing websocket/TCP

Il relay pair proxy:

- accoppia i due lati del relay tramite uuid
- supporta pairing websocket e TCP
- evita di dipendere completamente dal comportamento nativo di hbb 1.1.15

d. Fallback su 443

Risultato finale:

- se le porte standard sono raggiungibili, il client puo usarle
- se sono bloccate, UpDesk continua a funzionare via wss su 443

e. Loopback legacy errato

Bug corretto:

- alcune risposte legacy potevano suggerire 127.0.0.1: come target diretto
- il client ora ignora quel loopback come destinazione finale e forza il relay pubblico

3.4 Modifiche lato client per il relay

File chiave:

- src/client.rs
- src/rendezvous_mediator.rs
- libs/hbb_common/src/socket_client.rs

Modifiche consolidate:

- accettazione di fallback legacy coerenti
- gestione piu robusta del relay quando il server vecchio non parla esattamente il protocollo nuovo
- mantenimento del fallback websocket su 443 senza rompere il path standard

4. Nginx e deploy server

Il relay custom non vive solo nel codice: il fork include anche il materiale operativo per ripristinarlo.

File chiave:

- deploy_updesk_server.py
- server/updesk_nginx_patch.py
- backups/UPDESK-RELAY-SERVER-RECOVERY.md

Modifiche/strumenti:

- deploy automatico bridge/proxy sul VPS
- applicazione patch nginx per:

- websocket upgrade
- timeout lunghi
- proxy_buffering off
- enable/restart dei servizi systemd
- runbook di recovery in caso di perdita del server

5. Inclusione dei sorgenti server upstream

Per evitare dipendenze esterne non tracciate, il fork include una copia locale dei sorgenti server di riferimento.

Percorso:

- third_party/updesk-server-1.1.15

Scopo:

- avere nel progetto i sorgenti hbbs/hbbr usati come baseline
- poter leggere e confrontare il comportamento upstream
- non dipendere solo da riferimenti remoti o dal server già installato

Importante:

- questa cartella rappresenta la baseline upstream
- le modifiche compat del fork stanno invece nella cartella:
- server

6. Sistema di auto-update professionale

Seconda grande area di personalizzazione del fork.

Documentazione dedicata:

- docs/updesk-updater.md

6.1 Obiettivo

Implementare un primo updater professionale con:

- manifest remoto
- download dedicato
- verifica SHA256
- helper separato per lanciare l'installer
- integrazione UI
- pubblicazione via updesk.uptimeservice.it

6.2 URL update

Path pubblici:

- <https://updesk.uptimeservice.it/api/v1/update/stable.json>
- <https://updesk.uptimeservice.it/releases/windows/updesk-<version>.exe>

Canale aggiunto nel fork:

- stable
- predisposizione beta

6.3 File principali lato updater

Client Rust:

- src/common.rs
- src/updater.rs
- src/flutter_ffi.rs
- src/platform/windows.rs
- src/bin/updesk_updater.rs

UI Flutter:

- flutter/lib/utls/update_service.dart
- flutter/lib/common.dart
- flutter/lib/models/state_model.dart
- flutter/lib/new_ui/modern_home_page.dart
- flutter/lib/desktop/widgets/update_progress.dart

Deploy/publish:

- deploy_updesk_update_assets.py
- stable.json
- server/updesk-nginx-update.conf.example

6.4 Comportamento implementato

Il flusso attuale fa:

1. check del manifest remoto
2. confronto tra versione locale e remota
3. download in:
 - %TEMP%\updesk-update\
4. verifica SHA256
5. avvio updesk_updater.exe
6. chiusura dell app e lancio dell installer

6.5 Migliorie implementate nel fork

a. Client brandizzato abilitato agli update

Il controllo update non viene piu escluso per custom client/brand client.

b. Download robusto

Se il file locale manca, il flow:

- non fallisce in silenzio
- ricarica
- verifica I hash

c. Verifica hash obbligatoria

Nessun bypass del controllo SHA256.

d. Updater helper separato

updesk_updater.exe:

- attende la chiusura del processo
- puo forzare la chiusura se serve
- lancia l'installer
- prova a riavviare UpDesk
- scrive log dedicato in %TEMP%\updesk-update\updesk_updater.log

e. Fallback di lancio installer

Il ramo Windows:

- tenta l'avvio con --update
- se fallisce ritenta senza quell'argomento

f. Stato update condiviso Rust -> UI

Il fork espone e usa stati di update leggibili:

- checking
- available
- downloading
- verifying
- ready
- preparing
- launching
- installer-launched
- up-to-date
- failed

g. Coerenza versioning release

Lo script release e stato corretto per aggiornare insieme:

- Cargo.toml
- src/version.rs
- flutter/pubspec.yaml
- uptimesdesk_setup.iss
- uptimesdesk_full_setup.iss

Questo evita pacchetti etichettati come 1.0.x ma compilati con core Rust di una versione diversa.

7. UI update stile prodotto

Il fork ha spinto la gestione update oltre il solo popup tecnico.

7.1 Home moderna

File:

- flutter/lib/new_ui/modern_home_page.dart

Modifiche:

- banner update dedicato
- bottone Riavvia e installa
- pulsante Verifica aggiornamenti
- feedback di stato reale
- niente chiusura finestra alla cieca prima di sapere se l'update è partito

7.2 Prompt globale desktop

File:

- flutter/lib/common.dart

Modifiche:

- prompt update globale
- fallback ai dati cache del core Rust
- stesso ramo update-me usato dalla home moderna

7.3 Pagina impostazioni dedicata

File:

- flutter/lib/desktop/pages/desktop_setting_page.dart

Modifiche:

- nuova tab Updates
- policy chiare:
 - Disattivato
 - Canale stabile
 - Aggiornamento automatico consigliato
 - Canale beta
- stato leggibile:
 - versione attuale
 - versione remota
- manifest attivo
- changelog
- versione minima supportata
- stato/errore
- azioni:
 - Verifica aggiornamenti
 - Riavvia e installa

Questa parte è la modifica più vicina a un comportamento tipo AnyDesk .

8. Deploy e sincronizzazione locale del client

File:

- deploy_local_client.ps1

Modifiche/ruolo:

- stop processo uptimedesk
- copia:
- libuptimedesk.dll
- uptimedesk.exe
- data/
- updesk_updater.exe
- riavvio dell app installata

Perche e importante:

- senza questo riallineamento, si rischiava di testare una build Rust aggiornata con UI Flutter vecchia oppure viceversa

9. Script di release e publish

File principali:

- release.ps1
- deploy_updesk_update_assets.py

Modifiche principali:

- build DLL desktop
- build helper updater
- build Flutter release
- aggiornamento centralizzato della versione
- generazione stable.json
- calcolo hash SHA256
- publish automatico di manifest e installer
- verifica URL pubblici update

10. Recovery e sorgenti totali

Per non perdere il lavoro in caso di problemi sul server, il fork include anche materiale di recovery.

File e cartelle:

- backups/UPDESK-RELAY-SERVER-RECOVERY.md
- backups/updesk-relay-server-recovery-2026-05-16
- archivio zip di recovery in backups/

Questa parte non cambia il comportamento del prodotto, ma cambia la maturita operativa del fork:

- sorgenti compat inclusi
- sorgenti server upstream inclusi
- deploy documentato
- recovery documentata

11. Traduzioni e localizzazione

Evidenze:

- molte stringhe nei file:
- src/lang
- script:
- add_translations.ps1

Modifiche:

- sostituzione del brand nelle traduzioni
- aggiunta di testi legati a UpDesk
- riallineamento di messaggi installazione/update/sicurezza

12. Cosa NON è stato cambiato radicalmente

Per chiarezza, il fork non ha riscritto da zero questi blocchi:

- motore base RustDesk di remote control
- architettura fondamentale di streaming/input/capture
- hbbs/hbbr upstream vendorizzati come baseline
- dominio o topologia relay pubblica già in uso

La strategia del fork è stata:

- mantenere la base RustDesk dove possibile
- aggiungere layer compat e di prodotto solo nei punti necessari

13. Sintesi finale delle aree custom

Le modifiche del fork si possono riassumere così:

1. Rebranding completo

- nomi binari, crate, installer, bundle, testi e asset

2. Relay compat su 443

- bridge websocket rendezvous
- proxy TCP compat
- relay pair proxy
- nginx patchato

3. Compatibilità client nuovo / server vecchio

- traduzione protobuf
- gestione register/online/relay
- correzione dei fallback loopback errati

4. Auto-update professionale

- manifest remoto
- download
- verifica SHA256
- helper updater separato
- publish script

5. UI update commerciale

- banner

- prompt
- pagina impostazioni dedicata
- policy e stato leggibile

6. Maturita operativa

- deploy locale
- deploy server
- recovery
- sorgenti upstream inclusi

14. File piu importanti da conoscere

Se devi capire il fork velocemente, parti da qui:

- Cargo.toml
- src/common.rs
- src/updater.rs
- src/flutter_ffi.rs
- src/client.rs
- flutter/lib/new_ui/modern_home_page.dart
- flutter/lib/desktop/pages/desktop_setting_page.dart
- server/ws_hbbs_bridge.py
- server/hbbs_tcp_proxy.py
- server/relay_pair_proxy.py
- deploy_updesk_server.py
- deploy_updesk_update_assets.py
- deploy_local_client.ps1
- release.ps1
- docs/updesk-relay-compatible.md
- docs/updesk-updater.md

15. Raccomandazione operativa

Per mantenere il fork governabile:

- considera questo documento come l'indice principale delle personalizzazioni
- aggiorna sempre questo file quando aggiungi:
 - nuove policy update
 - nuove patch relay
 - nuovi componenti di deploy
 - nuove deviazioni importanti dalla base RustDesk